

[BE-Monitoring]Grafana + Prometheus

1. SpringBoot 서버셋팅

1-1 실습 프로젝트 생성

기본 패키지 추가후

아래 소스 참고하여

2가지 패키지 추가해주기

[build.gradle]

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '3.5.6'  
    id 'io.spring.dependency-management' version '1.1.7'  
}  
  
group = 'com.example'  
description = 'Demo project for Spring Boot'  
  
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(21)  
    }  
}  
  
configurations {  
    compileOnly {  
        resolutionStrategy {  
            force 'org.springframework:spring-core:5.3.20'        }  
    }  
}
```

```

        extendsFrom annotationProcessor
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-web'

    # monitoring
    implementation 'org.springframework.boot:spring-boot-starter-actuator'
    implementation 'io.micrometer:micrometer-registry-prometheus'

    compileOnly 'org.projectlombok:lombok'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    runtimeOnly 'org.mariadb.jdbc:mariadb-java-client'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.named('test') {
    useJUnitPlatform()
}

```

테스트용 Path 하나 만들어둔다.

/test

1-2 Grafana, Prometheus 용 도커 파일 셋팅

[docker-compose.yml]

```
version: "3.8"

services:
  springboot:
    image: openjdk:21-jdk-slim
    container_name: springboot-app-rev2
    working_dir: /app
    volumes:
      - ./build/libs:/app
    command: ["java", "-jar", "demo.jar"]
    ports:
      - "8085:8085"
    networks:
      - monitoring

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
    networks:
      - monitoring

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin
    networks:
      - monitoring
```

```
networks:
  monitoring:
    driver: bridge
```

[application.yml]

```
spring:
  datasource:
    url: jdbc:mariadb://컨테이너명:3306/디비명
    username: root
    password: 1234
    driver-class-name: org.mariadb.jdbc.Driver

  jpa:
    hibernate:
      ddl-auto: create-drop # or 'create', 'validate', etc.
    show-sql: true
    properties:
      hibernate:
        format_sql: true
    database-platform: org.hibernate.dialect.MariaDBDialect

  server:
    port: 8085

  management:
    endpoints:
      web:
        exposure:
          include: health, prometheus
    endpoint:
      health:
        show-details: always
```

[prometheus.yml]

```
global:
  scrape_interval: 5s # 5초마다 수집

scrape_configs:
  - job_name: 'springboot-app'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['springboot-app-rev2:8085']
```

- >  .gradle
- >  build
- >  gradle ✓
- ✓  prometheus
 -  prometheus.yml
- >  src
 -  .gitattributes
 -  .gitignore
 -  build.gradle
 -  deploy.sh
 -  docker-compose.yml ✓
 -  dockerfile
 -  gradlew
 -  gradlew.bat
 -  settings.gradle

1-3 도커 컨테이너 올리기

```
docker compose up -d
```

1-4 컨테이너 올린 부트 테스트 api 날리기

```
http://localhost:8085/test
```

컨테이너 에러 발생시 원인 찾기

```
docker logs -f 컨테이너명
```

2. Prometheus 확인

Prometheus는 서버의 리소스 데이터를 긁어오고
긁어온 데이터를 Grafana가 시각화 해준다.

2-1 서버 리소스 자원 확인해보기

```
http://localhost:8085/actuator/prometheus
```

```

localhost:8085/actuator/prometheus

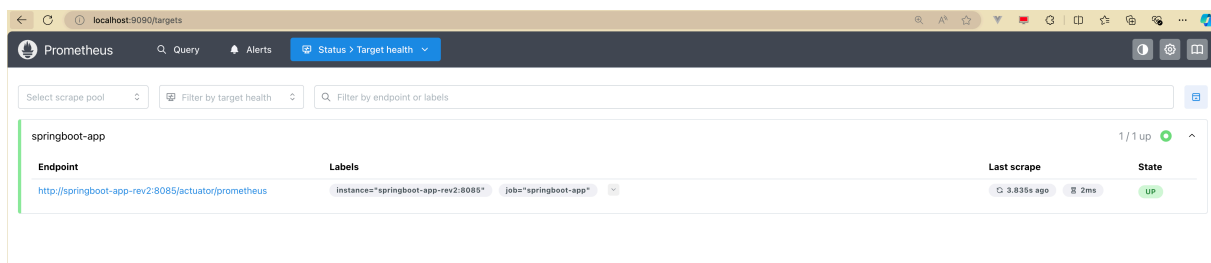
# HELP application_ready_time_seconds Time taken for the application to be ready to service requests
# TYPE application_ready_time_seconds gauge
application_ready_time_seconds{main_application_class="com.example.demo.DemoApplication"} 2.177
# HELP application_started_time_seconds Time taken to start the application
# TYPE application_started_time_seconds gauge
application_started_time_seconds{main_application_class="com.example.demo.DemoApplication"} 2.128
# HELP disk_free_bytes Usable space for path
# TYPE disk_free_bytes gauge
disk_free_bytes{path="/app/."} 6.15178010624E11
# HELP disk_total_bytes Total space for path
# TYPE disk_total_bytes gauge
disk_total_bytes{path="/app/."} 9.9466258432E11
# HELP executor_active_threads The approximate number of threads that are actively executing tasks
# TYPE executor_active_threads gauge
executor_active_threads{name="applicationTaskExecutor"} 0.0
# HELP executor_completed_tasks_total The approximate total number of tasks that have completed execution
# TYPE executor_completed_tasks_total counter
executor_completed_tasks_total{name="applicationTaskExecutor"} 0.0
# HELP executor_pool_core_threads The core number of threads for the pool
# TYPE executor_pool_core_threads gauge
executor_pool_core_threads{name="applicationTaskExecutor"} 8.0
# HELP executor_pool_max_threads The maximum allowed number of threads in the pool
# TYPE executor_pool_max_threads gauge
executor_pool_max_threads{name="applicationTaskExecutor"} 2.147483647E9
# HELP executor_pool_size_threads The current number of threads in the pool
# TYPE executor_pool_size_threads gauge

```

2-2 Prometheus 쿼리날려보기

<http://localhost:9090/>

http_server_requests_seconds_count



Prometheus Query Editor interface showing a query: `rate(http_server_requests_seconds_count[1m])`. The results are displayed in a table with 3 columns: Evaluation time, Load time: 3ms, and Result series: 3. The table contains 3 rows of data, each representing a different instance of the 'springboot-app' with various status codes (404, 200, 200). At the bottom, there is a '+ Add query' button.

여기까지 되면

데이터 굽어올 준비 끝

3. Grafana 셋팅

`http://localhost:3000`

`id: admin`

`pw: admin`



Welcome to Grafana

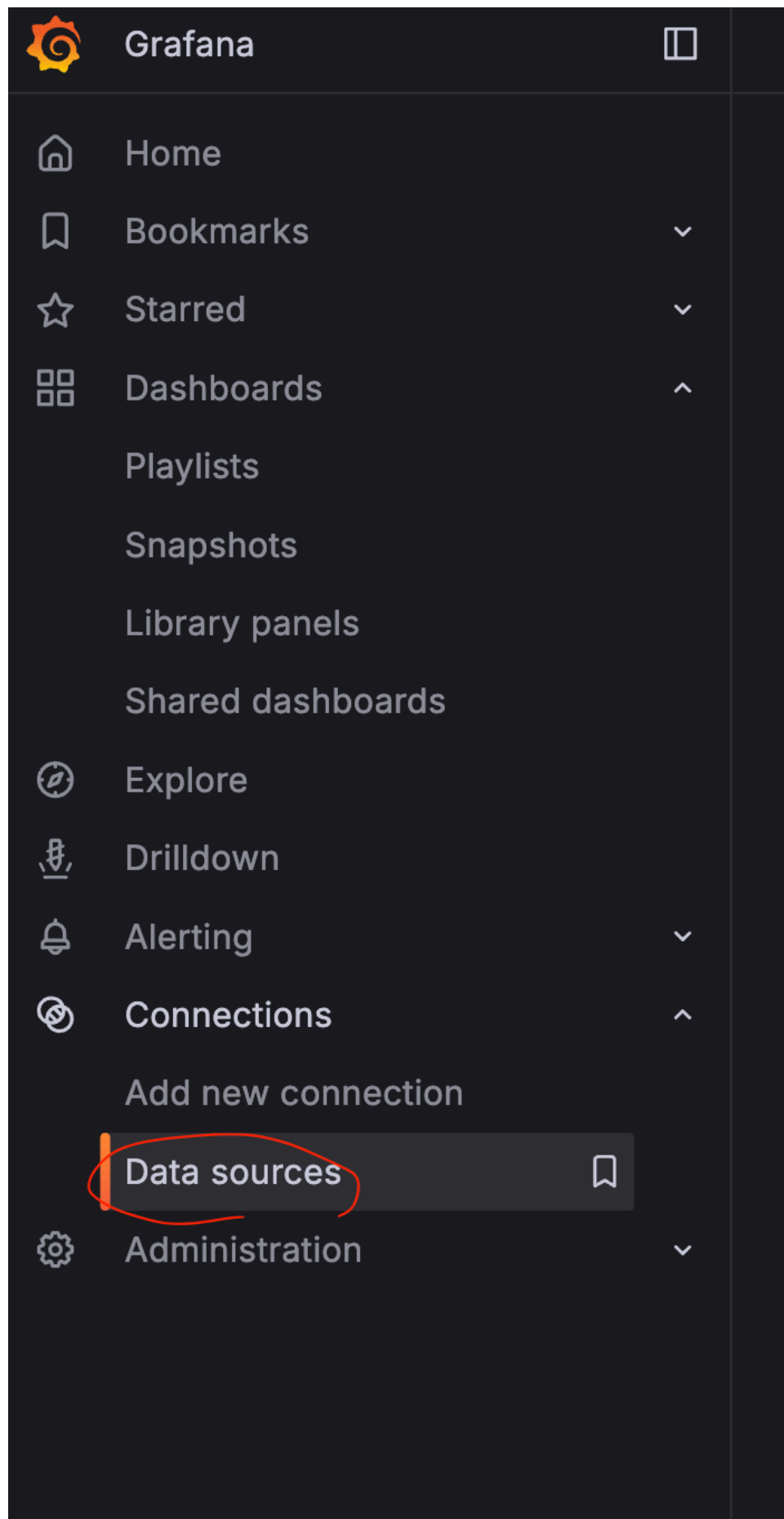
Email or username

Password



Log in

[Forgot your password?](#)



Home > Connections > Data sources

Q Search...

Data sources

View and manage your connected data source connections

Q Search by name or type

+ Add new data source

Sort by A-Z

Add data source

Choose a data source type

Q Filter by name or type

Time series databases



Prometheus
Open source time series database & alerting

Core

 **prometheus**

Type: Prometheus

Settings Dashboards

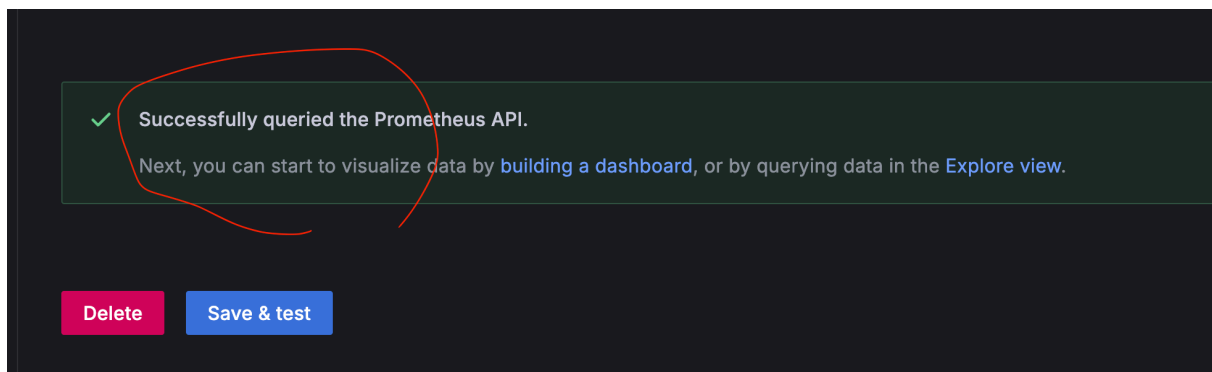
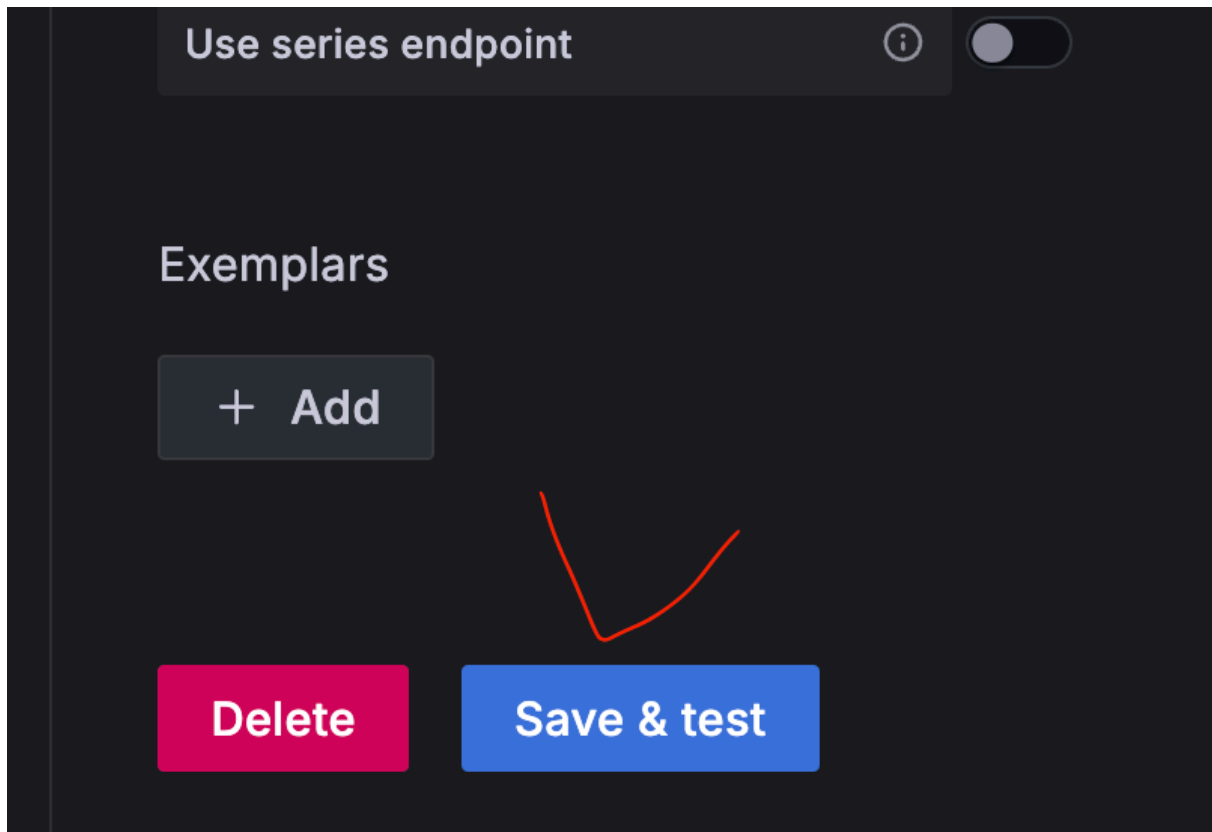
Name prometheus Default ☒

Before you can use the Prometheus data source, you must configure it below or in the config file. For detailed information see the Prometheus documentation.

Fields marked with * are required

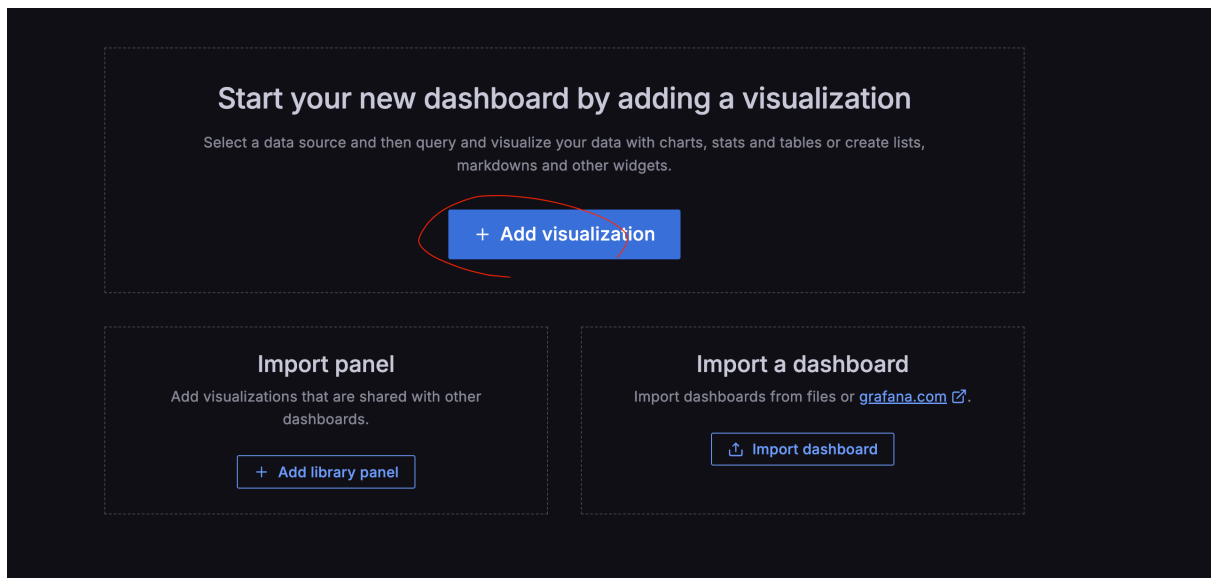
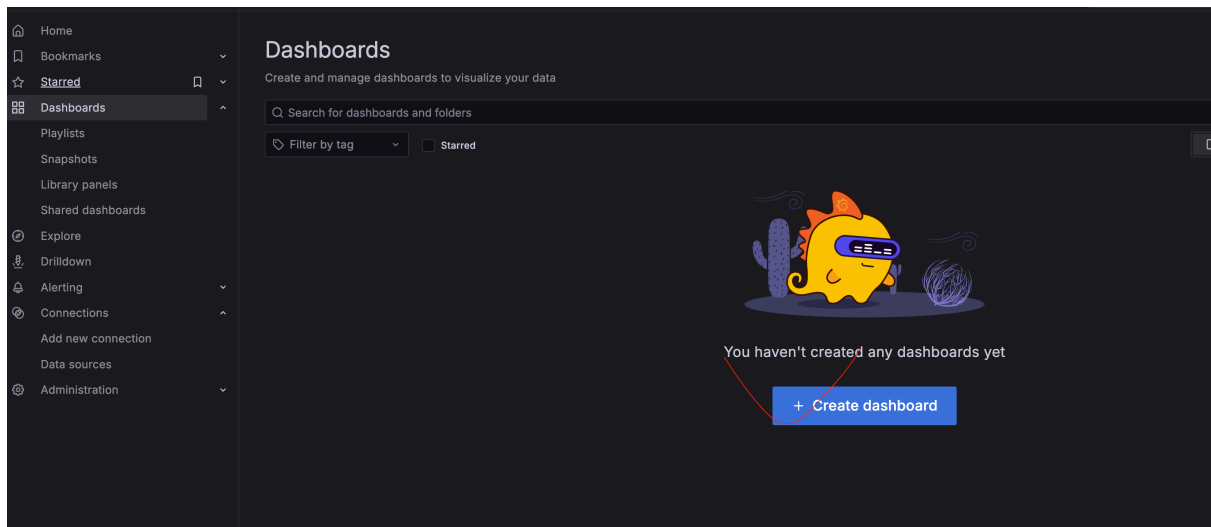
Connection

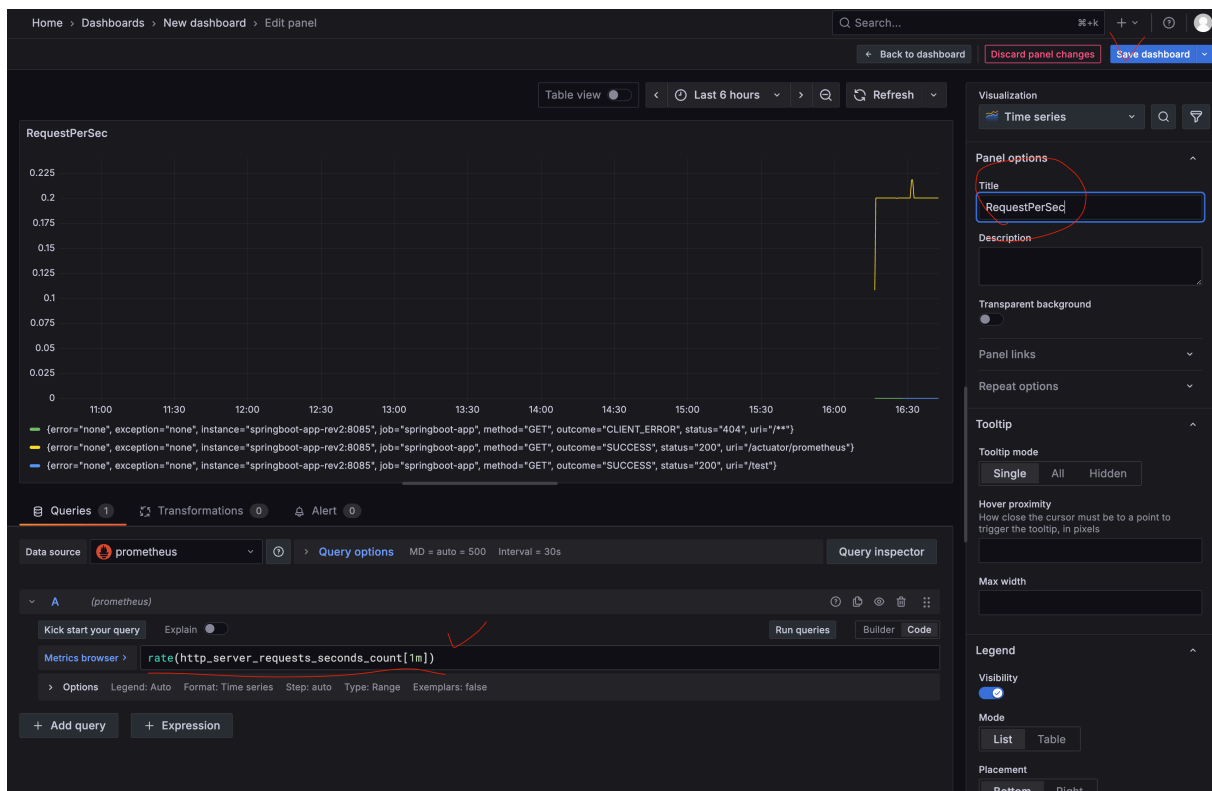
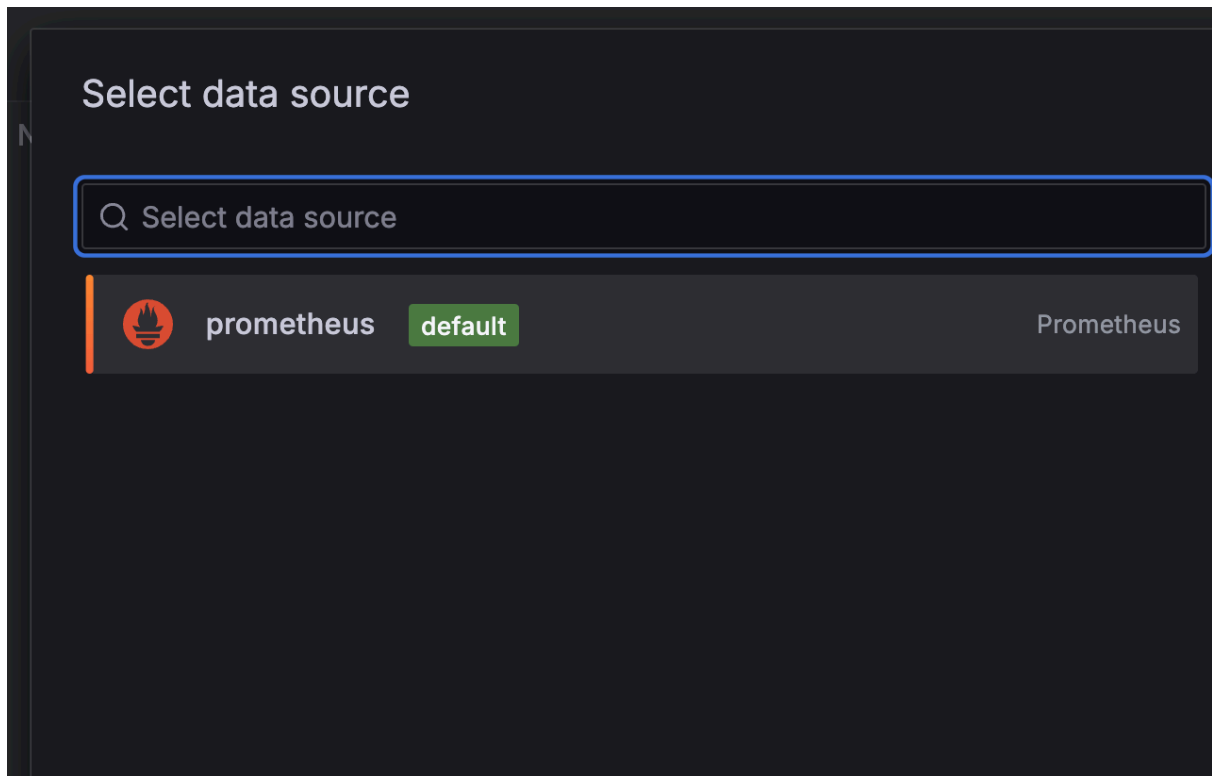
Prometheus server URL *



Grafana + Prometheus 체크완료

4. Grafana Dash보드 생성하기





Save dashboard

New dashboard

Details

Changes

7

Title

Monitor-Spring

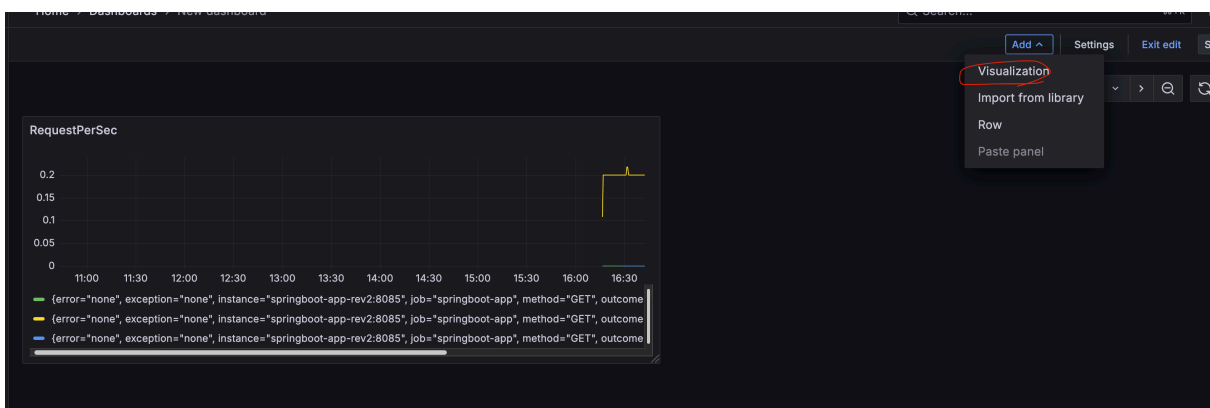
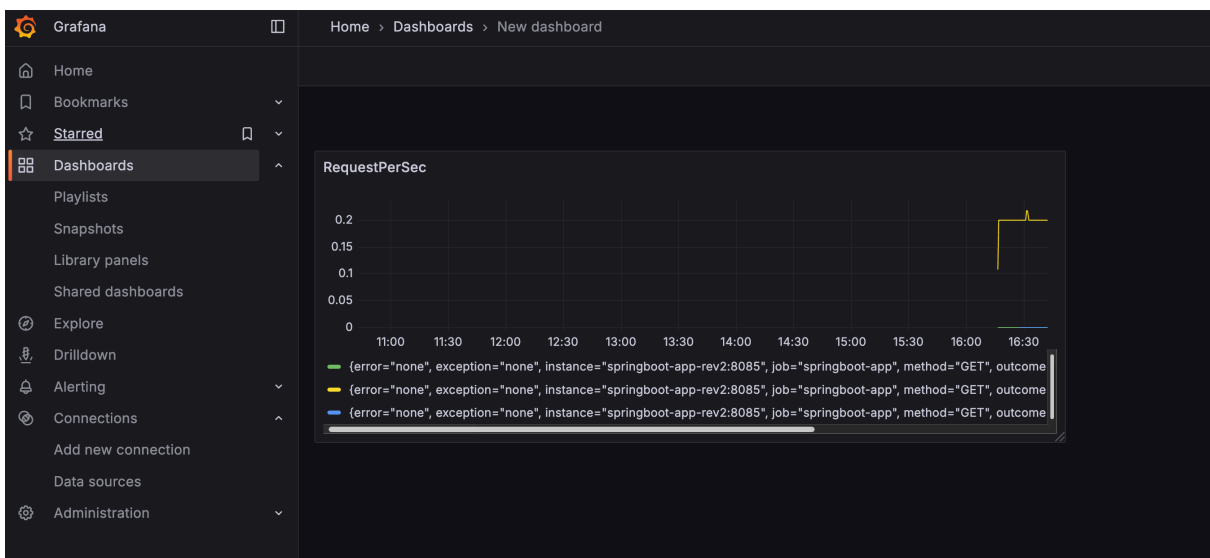
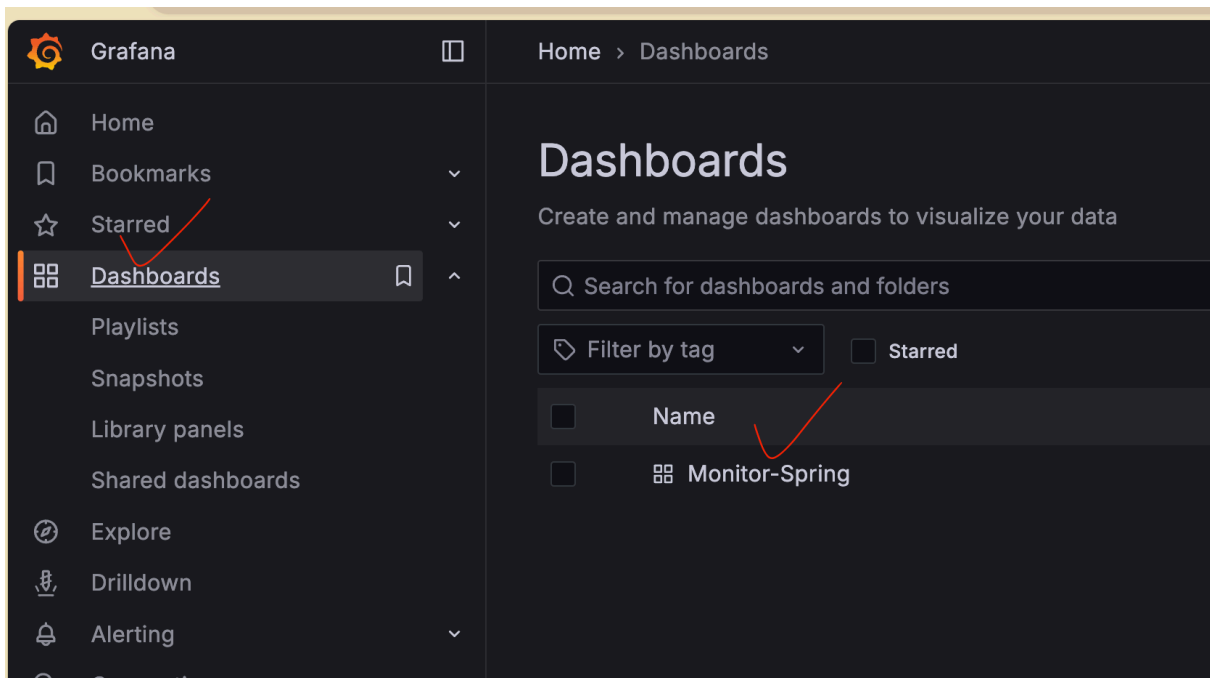
Description

Folder

📁 Dashboards

Cancel

Save



여러개 추가할 수 있다.

쿼리를 참고하여 원하는 패널 추가해보자

http_server_requests_seconds_count HTTP 요청 횟수

http_server_requests_seconds_sum 요청 처리 시간 총합

jvm_memory_used_bytes JVM 메모리 사용량

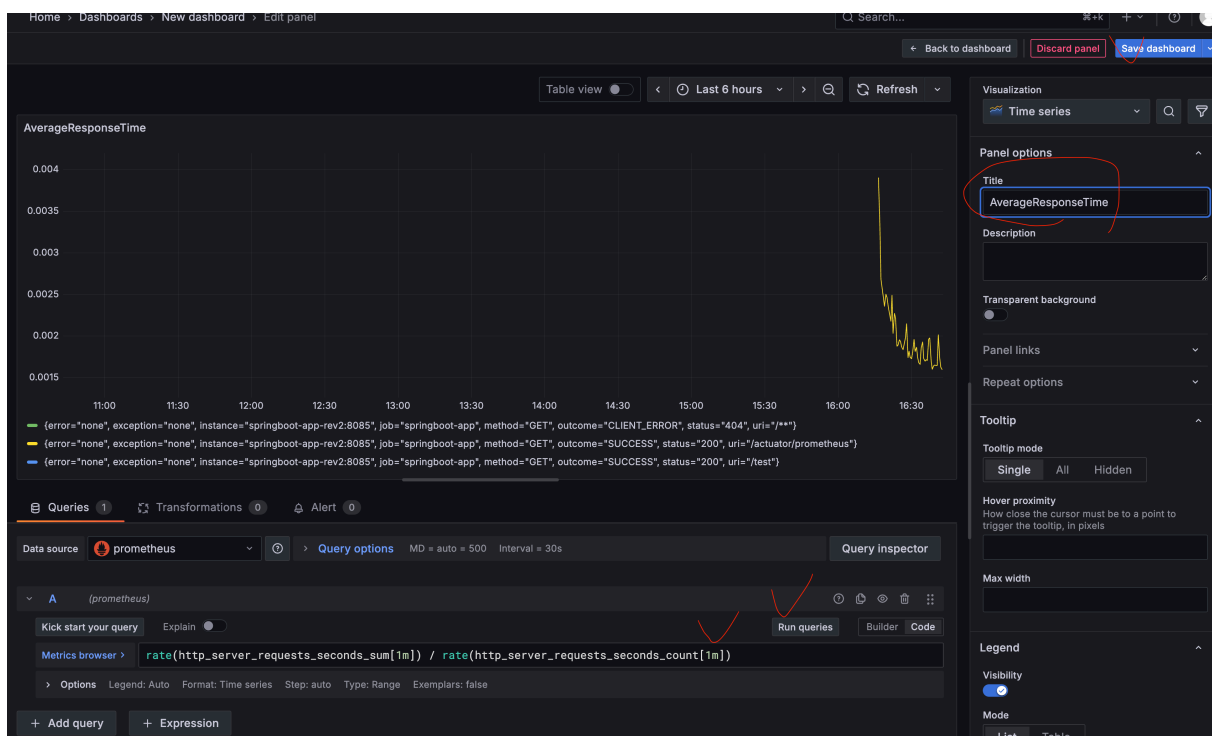
process_cpu_usage 애플리케이션 CPU 사용률

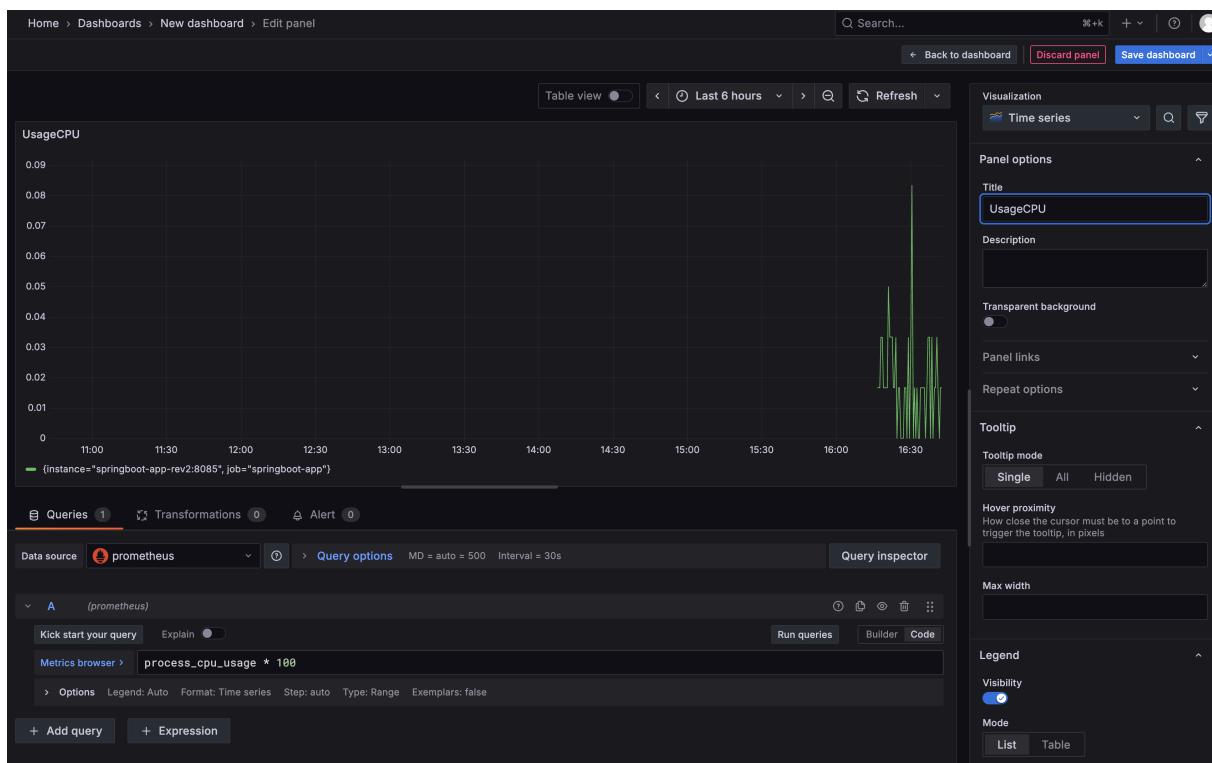
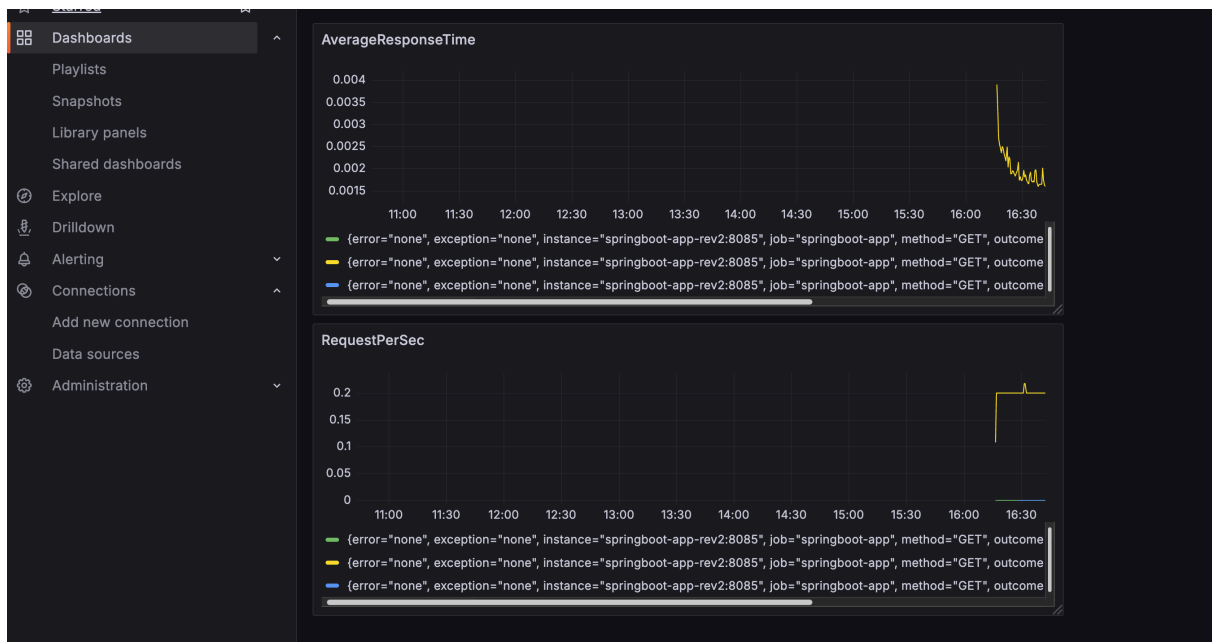
system_cpu_usage 전체 시스템 CPU 사용률

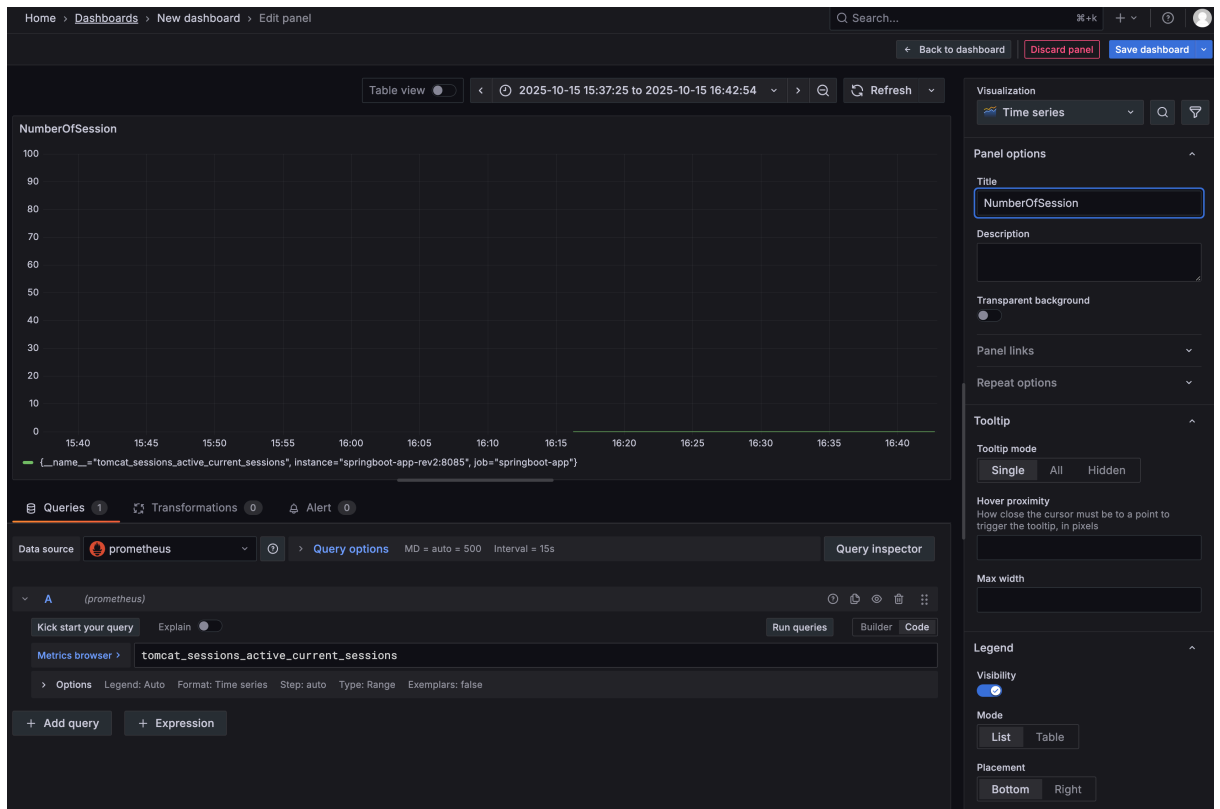
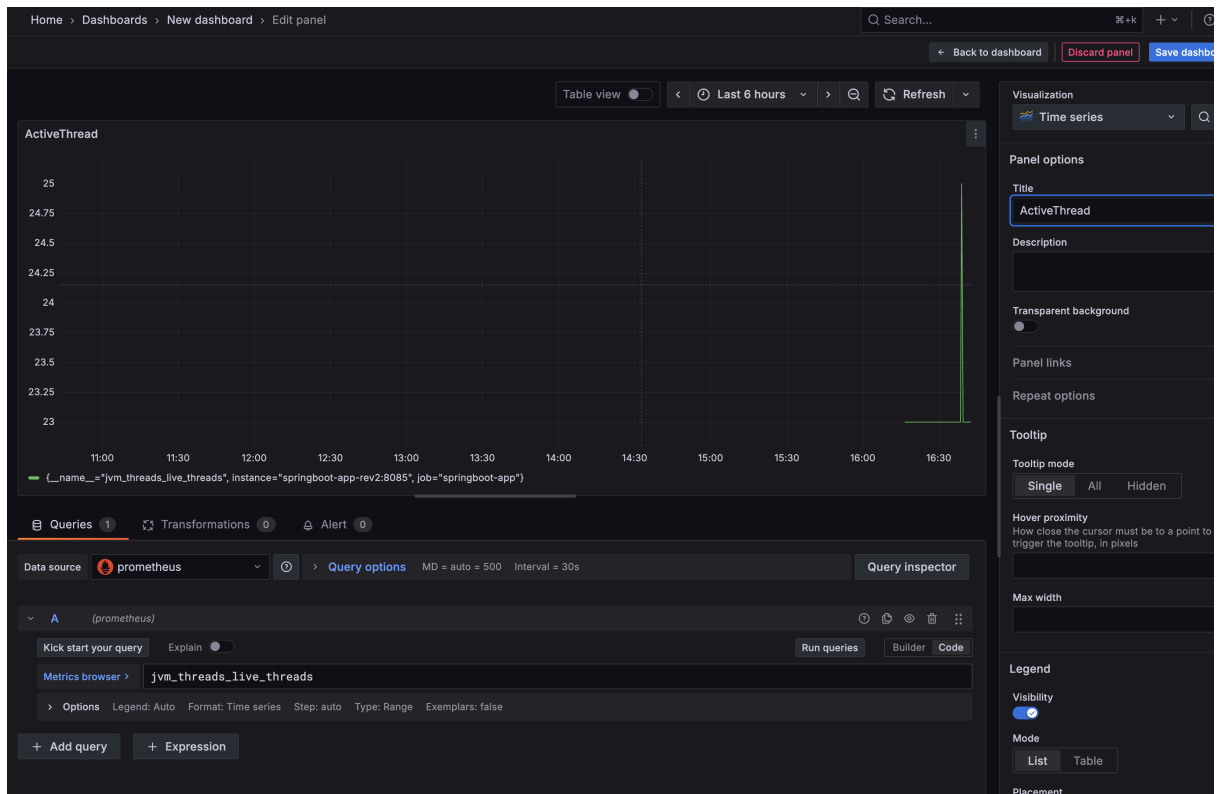
jvm_threads_live_threads 현재 살아있는 스레드 수

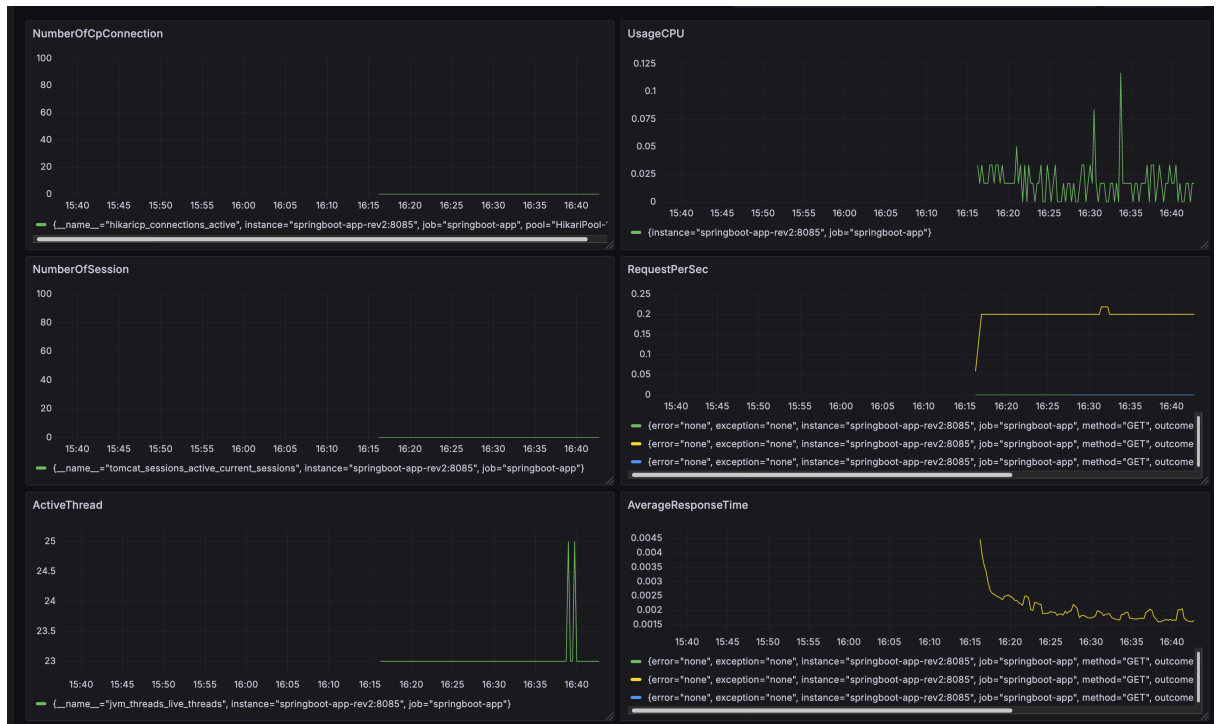
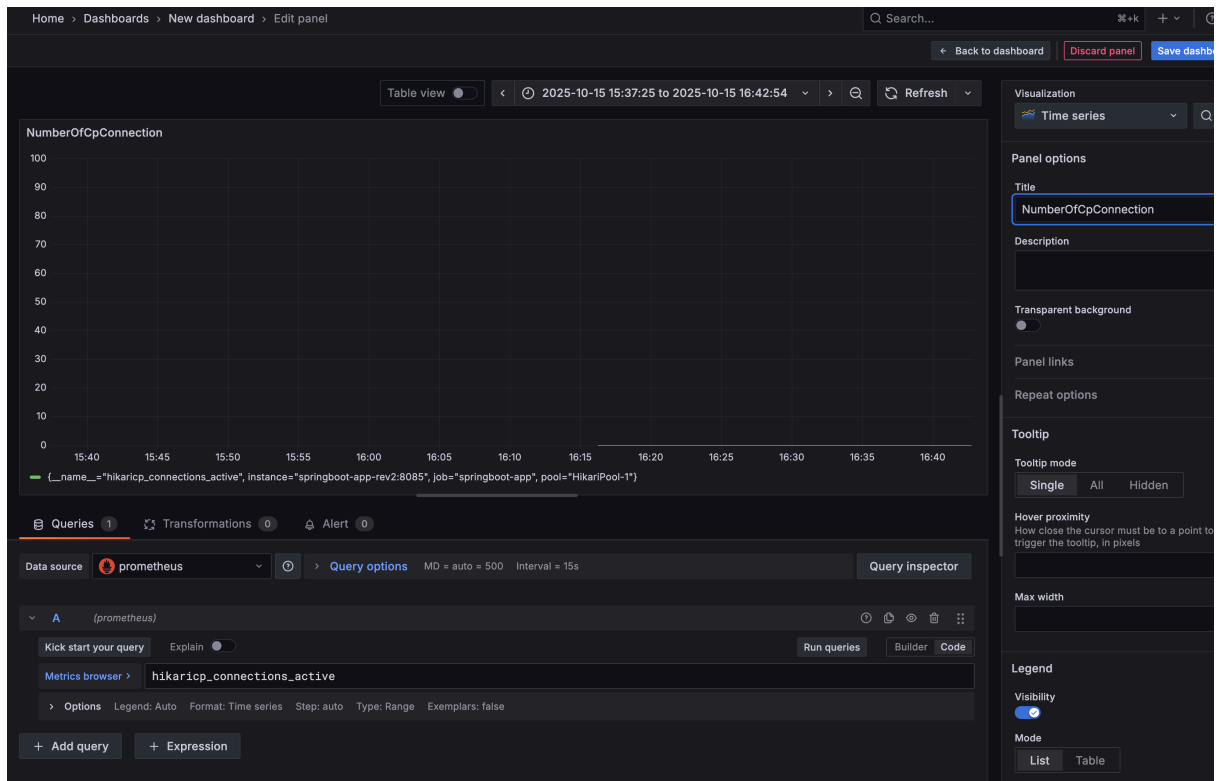
tomcat_sessions_active_current_sessions 활성 세션 수

hikaricp_connections_active 커넥션 사용수











그래프 드래그하여 특정 영역 확인 가능

테스트

요청 날려서 그래프 변화를 본다.